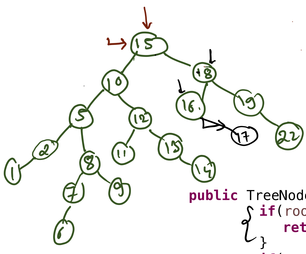
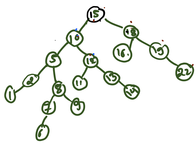
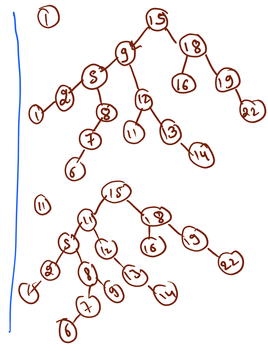
Lec-33

17/10/26 1:18 PM

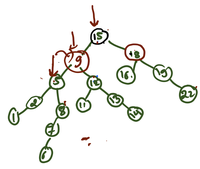


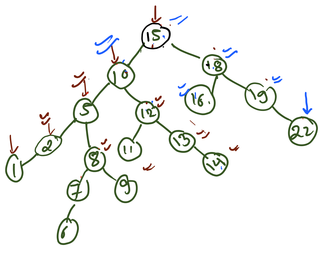
```
public TreeNode insertIntoBST(TreeNode root, int val) {
        if(root==null) {
            return new TreeNode(val);
        }
        if(root.val<val) {
            root.right=insertIntoBST(root.right, val);
        }
        else {
            root.left=insertIntoBST(root.left, val);
        }
        return root;
    }
```

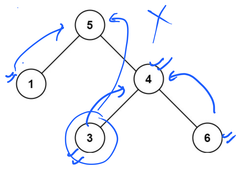


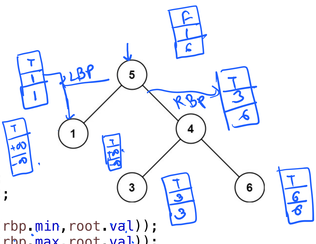


```
public TreeNode deleteNode(TreeNode root, int key) {
        if (root == null) {
            return null;
        }
        if (root.val < key) {
            root.right = deleteNode(root.right, key);
        } else if (root.val > key) {
            root.left = deleteNode(root.left, key);
        } else {
            // 0 and 1 child
            if (root.left == null) {
                return root.right;
            } else if (root.right == null) {
                return root.left;
            } else {
                int max=max(root.left);
                root.left = deleteNode(root.left, max);
                root.val=max;
            }
        }
        return root;
    }
```









```
public BstPair ValidBST(TreeNode root) {
        if(root==null) {
            return new BstPair();
        }
        BstPair lbp = ValidBST(root.left);
        BstPair rbp = ValidBST(root.right);
        BstPair sbp = new BstPair();
        sbp.min=Math.min(lbp.min,Math.min(rbp.min,root.val));
        sbp.max=Math.max(lbp.max,Math.max(rbp.max,root.val));
        if(lbp.bst && rbp.bst && lbp.max<root.val&& rbp.min>root.val) {
            sbp.bst=true;
        }
        else {
            sbp.bst=false;
        }
```

